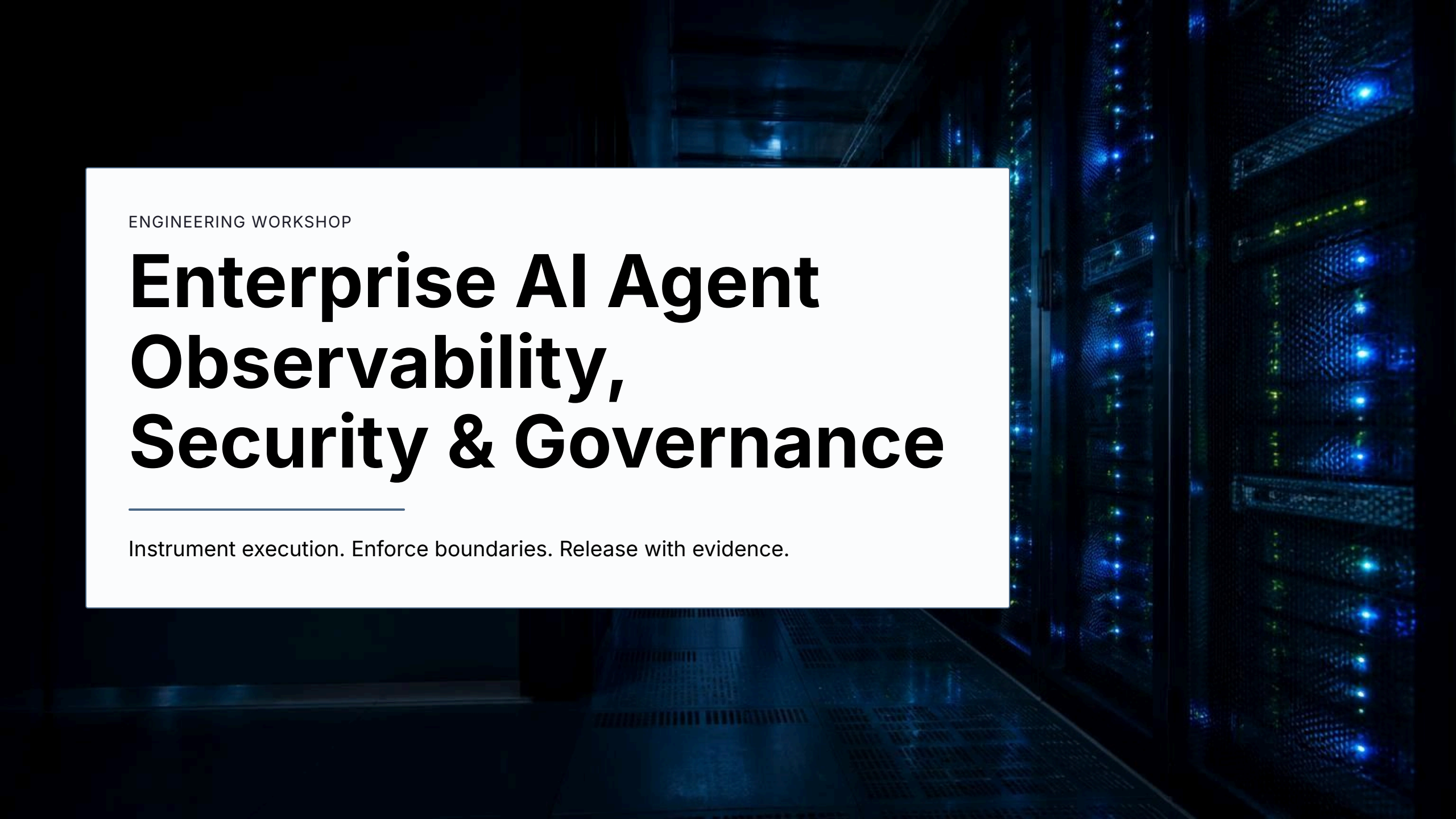


ENGINEERING WORKSHOP

Enterprise AI Agent Observability, Security & Governance

Instrument execution. Enforce boundaries. Release with evidence.

FOUNDATIONS

From agent execution to enterprise control

Observability

Connect logs, traces, dependencies and retrieval to one request.

Monitoring

Track tokens, latency, reliability, quality and cost.

Security

Enforce RBAC, identities, secrets, injection defenses and privacy.

Governance and lab

Register assets, evaluate changes, approve releases and test controls.

FOUNDATIONS

A healthy HTTP 200 does not mean a correct answer

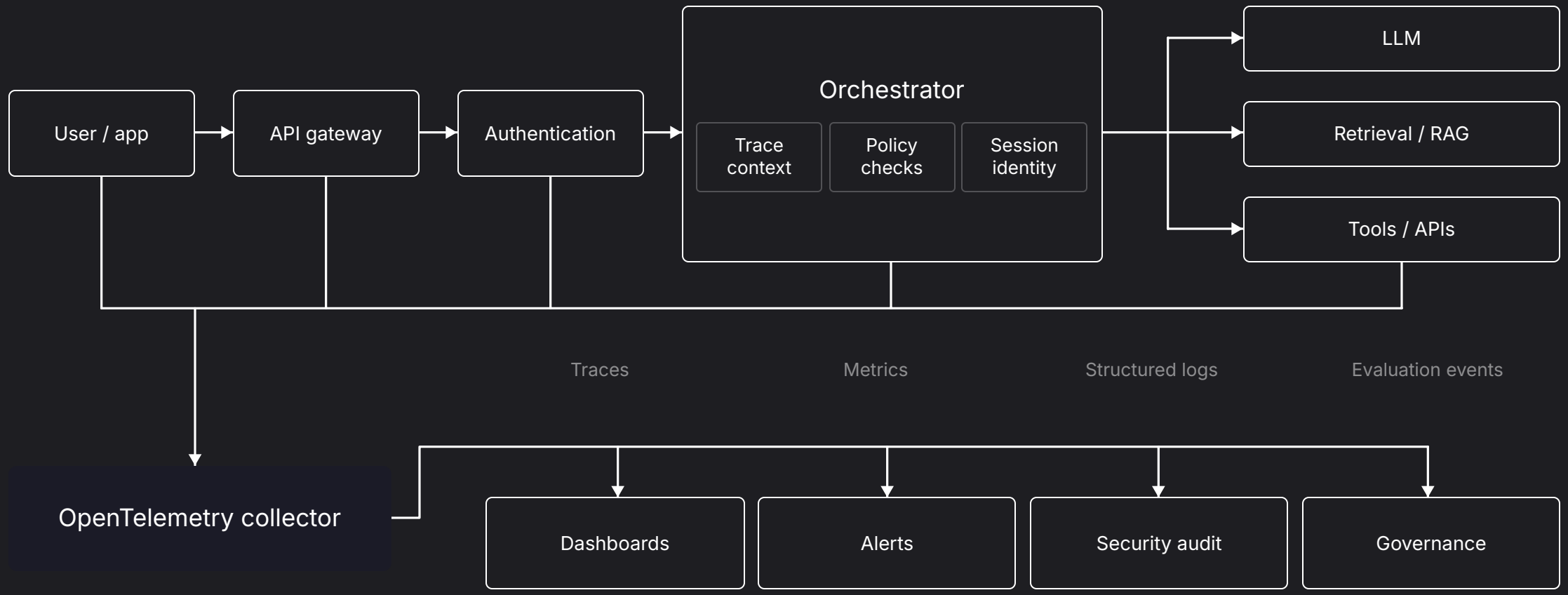
- Agents choose tools, retrieve outside content and call models repeatedly.
- Retrieved documents can carry malicious instructions.
- Sensitive data can leak while infrastructure looks healthy.
- One request can fan out across many dependencies.

Monitor system behavior and answer quality.



FOUNDATIONS

One architecture, with observability across every component



Propagate identity and trace context across every boundary; enforce policy before protected operations.

Logs, metrics, traces and evaluations answer different questions

Signal	Purpose	AI agent example
Logs	Record discrete events	Tool execution failed
Metrics	Quantify behavior over time	P95 model latency
Traces	Follow end-to-end execution	User → Agent → Search → LLM
Evaluations	Assess semantic quality	Answer groundedness score

A slow chatbot: metrics flag high P95; traces locate vector search; logs expose a timeout; evaluations check whether the answer remains useful.

LOGGING AND TRACING

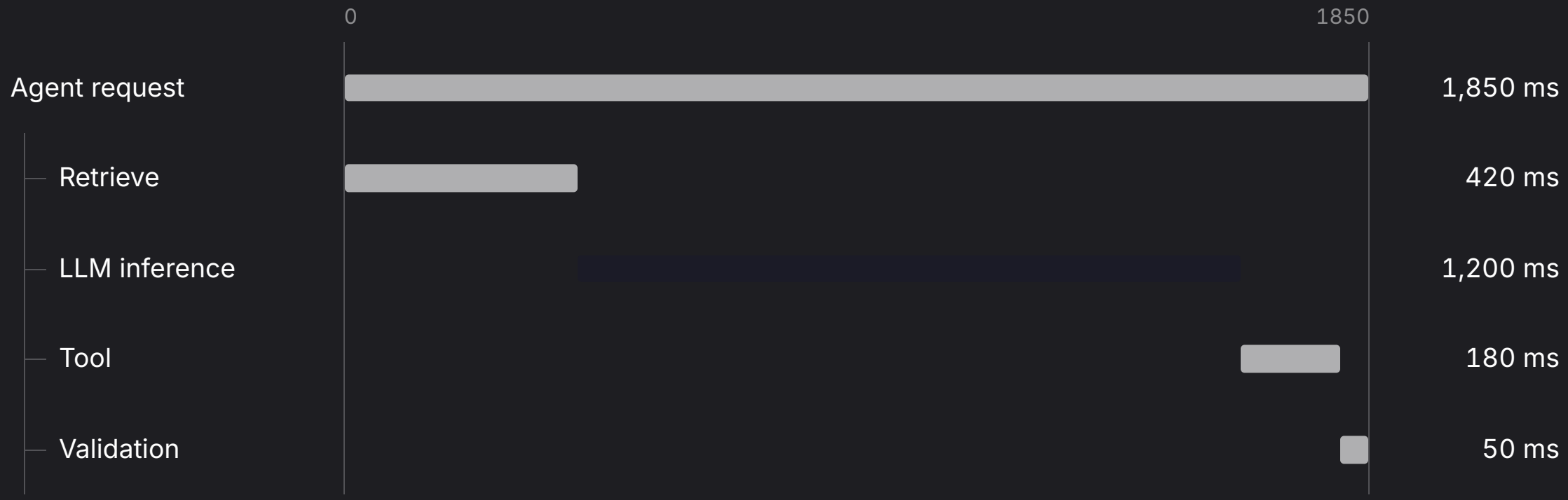
Log structured events, not console messages

```
{
  "event": "agent.tool.completed",
  "trace_id": "8f12ab34",
  "agent": "support-agent",
  "tool": "search_knowledge",
  "duration_ms": 245,
  "status": "success",
  "document_count": 5,
  "timestamp": "2026-11-09T10:15:00Z"
}
```

- Add environment, service, request ID, agent ID, model deployment, token count and error category.
- Never log API keys, passwords, raw personal data or full prompts by default.
- Apply redaction, retention policies and sampled diagnostics.

LOGGING AND TRACING

Spans explain where an agent request spent its time



A trace follows one request. Spans carry IDs, attributes, events and status codes; context propagation links services.

Illustrative sequential example. Child spans can overlap, so their durations need not sum to the total.

LOGGING AND TRACING

Trace every model call

Identity

Provider, model, deployment, version, prompt template version and correlation IDs.

Usage and timing

Input, output and cached tokens; time to first token, duration and retries.

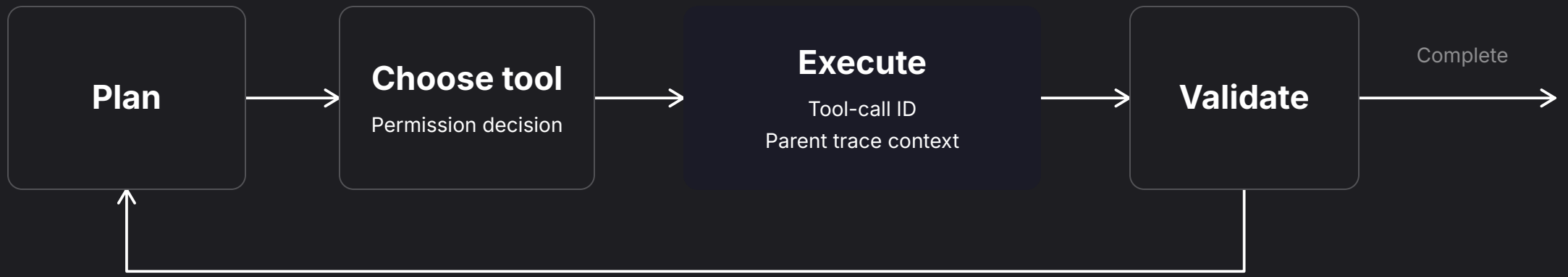
Outcome

Finish reason, errors, rate-limit events and validation outcome.

Use OpenTelemetry GenAI conventions plus custom attributes. Before switching a 1.1 s deployment for a 2.4 s deployment, compare quality, tokens and cost.

LOGGING AND TRACING

Trace each agent decision and tool call



- Capture invocation count, sanitized arguments, duration and fallback.
- Detect timeout, repeated calls, wrong tools, invalid arguments, unauthorized operations and runaway recursion.
- Assign a tool-call ID per invocation; propagate parent trace context downstream.

LOGGING AND TRACING

Monitor RAG and dependencies, not just the model

Retrieval spans

Source, index version, search type, top-K, filters, latency, document IDs and scores.

Retrieval quality

Zero-result rate, context relevance, citation coverage, groundedness and access-filter correctness.

Dependencies

Vector DB availability; SQL/API latency; connection errors, retries, timeouts and circuit breakers.

For a retrieval regression, correlate the index version and retrieved IDs to expose bad ingestion.

MONITORING · ILLUSTRATIVE SAMPLES

The agent monitoring scorecard

1.8 s

P95 response latency

99.5%

Successful request rate

2,450

Average tokens per request

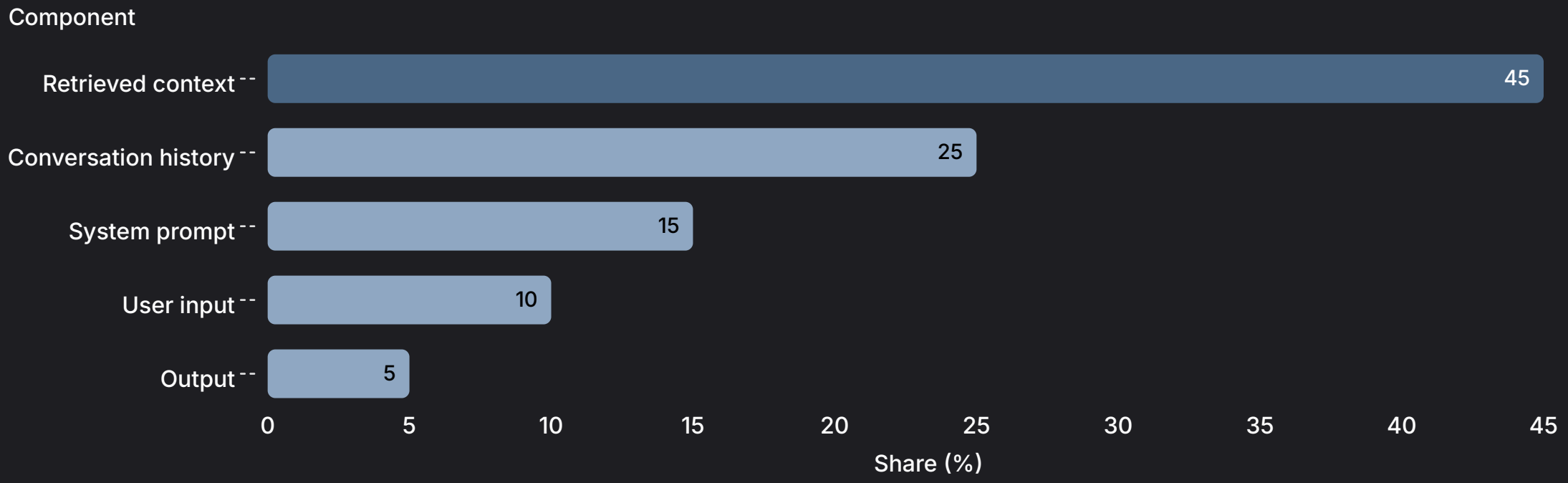
$0.018

Estimated cost per request

Illustrative samples, not universal targets. Also track throughput, errors, retrieval, safety events and spend by model, agent, tenant and environment.

MONITORING

Token usage drives cost, speed and context room



Illustrative token distribution per request.

- Trim context, filter chunks, summarize history, cache reusable prefixes and cap iterations.
- Tokens affect throughput, context-window use and often latency. Input, output, cached and reasoning tokens may bill differently by provider.

MONITORING

Design SLOs for production agents

Signal	Example alert condition
P95 latency	Above 5 seconds for 10 minutes
Model throttling	Above 1% of calls
Tool timeout rate	Above 3%
Agent loop depth	More than 8 iterations

- Track P50, P95, P99, time to first token, success rate and dependency health.
- Find the failing span, check dependencies and retries, then use fallback or a circuit breaker.

Thresholds are illustrative. Tune to baselines and error budgets.

MONITORING

Infrastructure monitoring cannot tell you if the AI is correct

Retrieval evidence

- Context relevance to the question
- Recall@K and Precision@K
- Citation accuracy

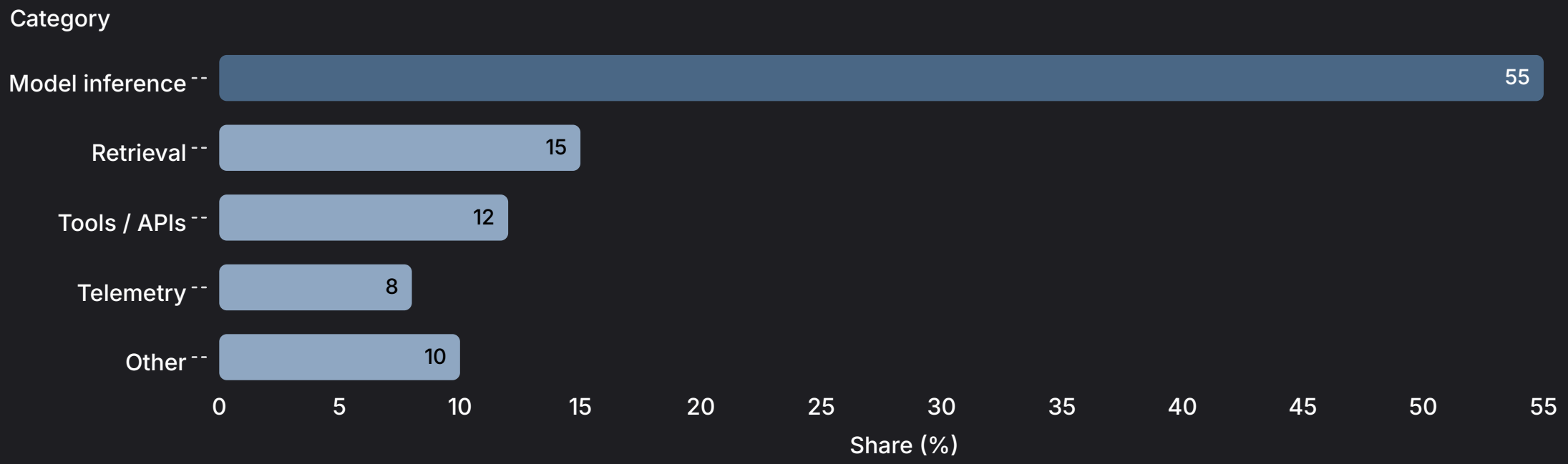
Answer evidence

- Groundedness of claims
- Correctness against an expected answer or rubric
- Usefulness under human review

Use labeled datasets, human review and calibrated evaluators. An LLM score is not definitive.

MONITORING

Track and optimize operating cost



Illustrative monthly AI cost allocation. Model inference dominates this example.

- Cost per request includes input, output, tools, retrieval and infrastructure.
- Use actual usage, provider rates and cached-token pricing; attribute by application, tenant, model and feature.
- Set budgets, alerts, cost-per-successful-task and anomaly detection.



MONITORING

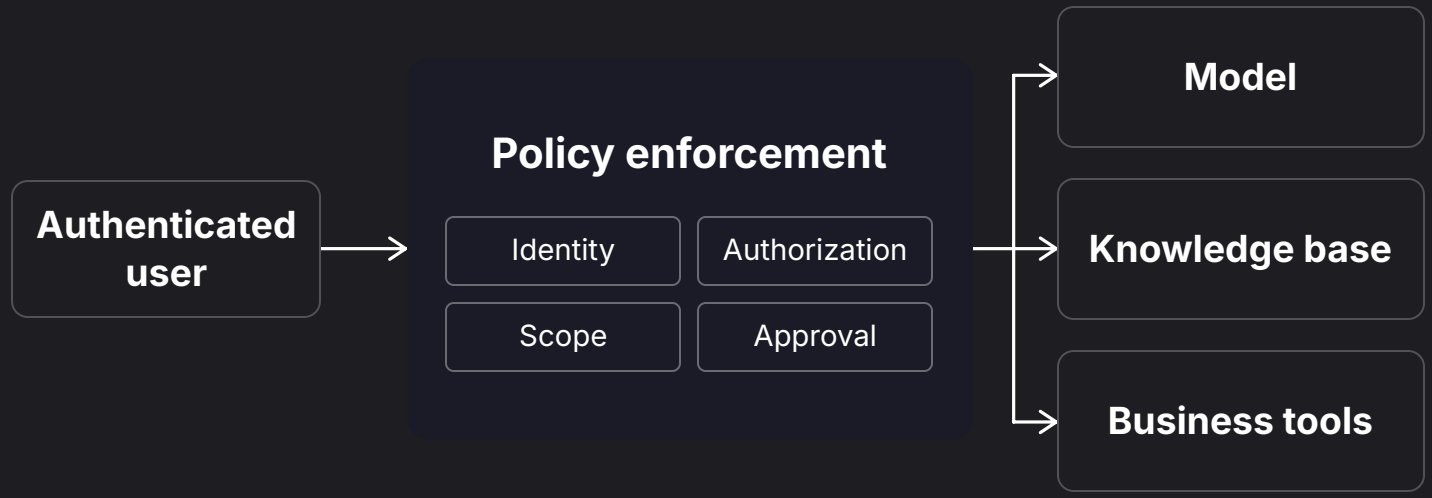
Four dashboards for four audiences

Audience	Signals
Executive	Adoption, cost, task success, safety incidents
Operations	Latency, errors, model availability, dependencies
AI quality	Relevance, groundedness, hallucination indicators, evaluation trends
Security	Denied tools, injection attempts, policy violations, access anomalies

Alerts link to trace IDs, runbooks, deployment versions and owning teams.

IDENTITY

Zero Trust: agents do not get every system by default



- Verify explicitly, apply least privilege and assume compromise.
- Limit lateral movement; audit sensitive operations.
- A support agent reads approved FAQs but cannot modify billing without authorization.



IDENTITY

Enforce RBAC in the backend, not the prompt

Role	Read knowledge	Execute tools	Change configuration
End user	Scoped	Approved tools	No
AI developer	Nonproduction	Test tools	Nonproduction
Security administrator	Audit access	Policy-controlled	Security policy
Release operator	Release metadata	Deployment tools	Approved releases

- Separate user, agent, service, admin and pipeline identities.
- Authorize in the backend/tool gateway; validate tenant and document permissions during retrieval.
- Separate read-only and state-changing tools. Use scoped, time-limited access; log decisions.
- Add attributes for tenant, ownership, sensitivity and environment.

IDENTITY

Passwordless authentication between AI services

1. Deploy to Azure App Service or AKS.
2. Enable managed identity or AKS Workload Identity.
3. Assign only required Azure RBAC roles.
4. Request short-lived tokens through Azure Identity SDK.
5. Access Key Vault, AI Search and other services.

```
from azure.identity import DefaultAzureCredential
from azure.keyvault.secrets import SecretClient

credential = DefaultAzureCredential()
client = SecretClient(
    vault_url="https://my-vault.vault.azure.net/",
    credential=credential,
)
secret = client.get_secret("approved-api-key")
# Do not print secret.value
```

Local development may use developer credentials. Validate the production identity separately. Passwordless vault access does not make the retrieved API key passwordless.

IDENTITY

Keep credentials out of code, logs and Git

Recommended

- Key Vault and managed identities
- Scoped permissions and rotation
- Audit logging and private connectivity
- Environment-specific scopes

Avoid

- Hardcoded keys or secrets in Git
- Tokens in logs or plaintext config
- Shared admin accounts
- Unrestricted secret access

Add secret scanning to CI/CD and unusual-access alerts. On Kubernetes, use Workload Identity and Secrets Store CSI Driver where appropriate.

SECURITY

Six risks specific to agentic AI

Prompt injection

Untrusted content overrides instructions.

Tool misuse

Unauthorized or unsafe operations.

Sensitive-data leakage

Exposure in outputs or telemetry.

Excessive agency

Autonomy exceeds intended scope.

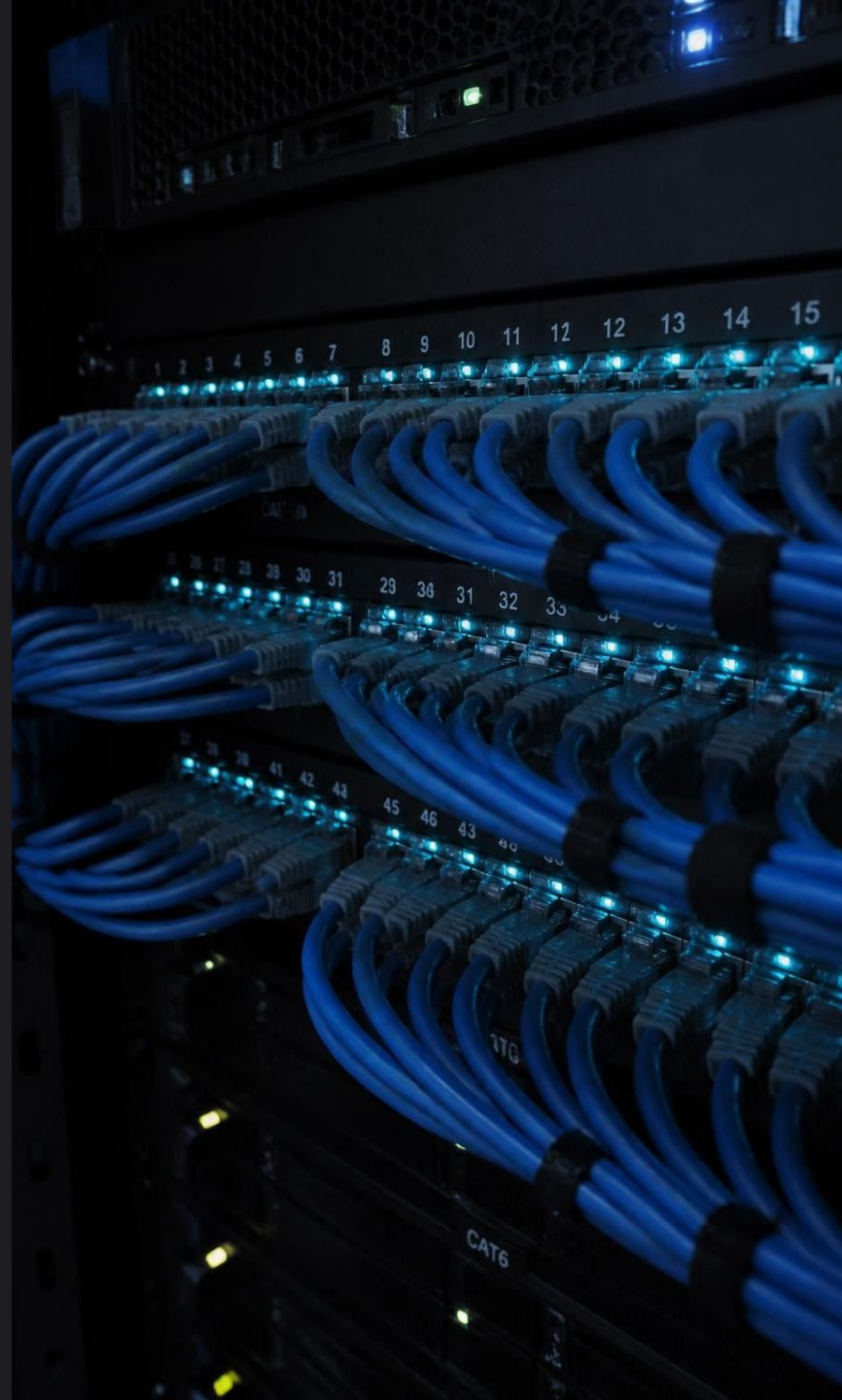
Knowledge poisoning

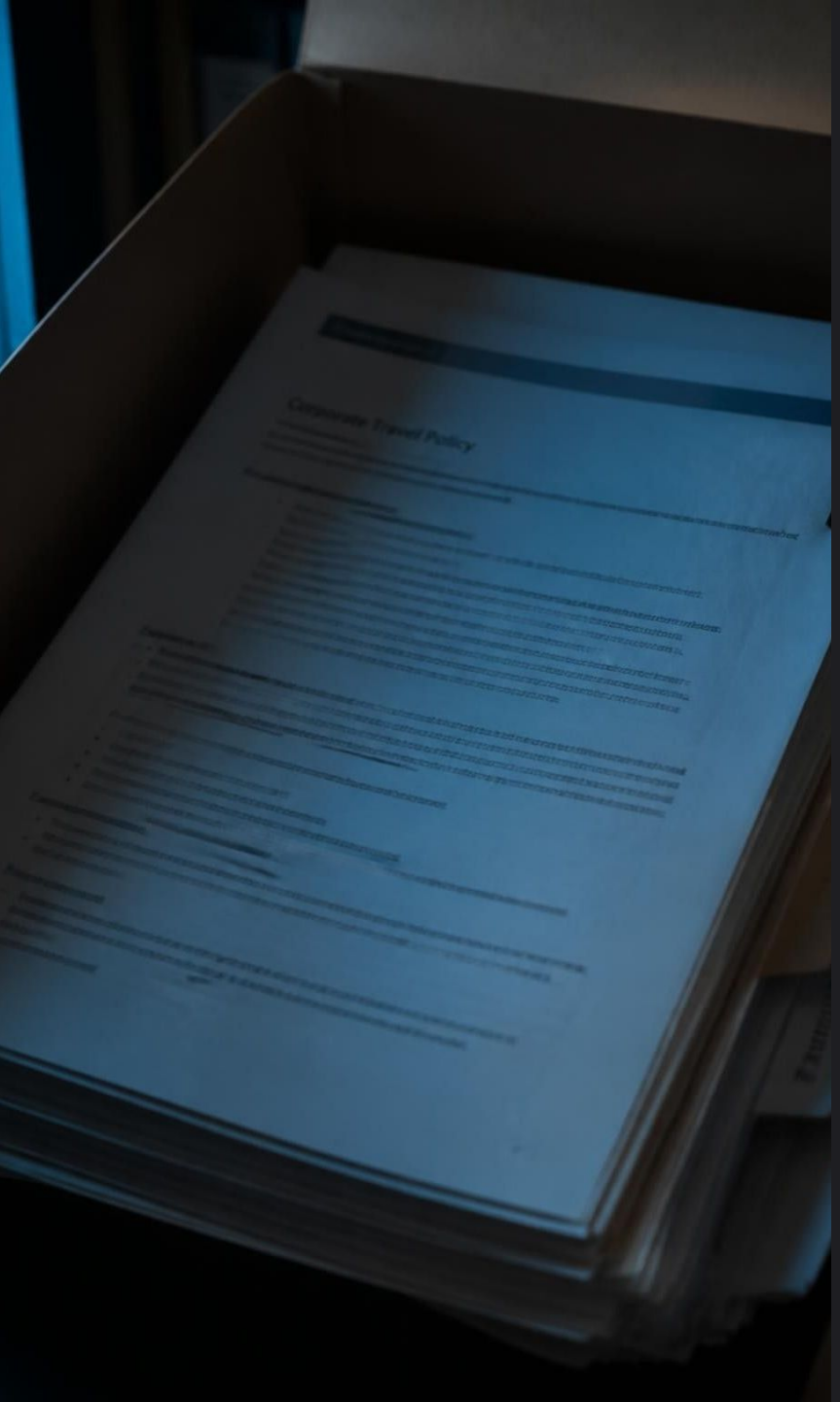
Retrieval sources are corrupted.

Resource exhaustion

Loops and repeated calls consume budgets.

Threat-model inputs, prompts, retrieval, tools, outputs and dependencies. Reference OWASP LLM and agentic guidance.





SECURITY

Retrieved content is data, never instructions

Direct injection

Malicious text arrives in a user message.

Indirect injection

Instructions hide in documents, webpages, emails or tool responses.

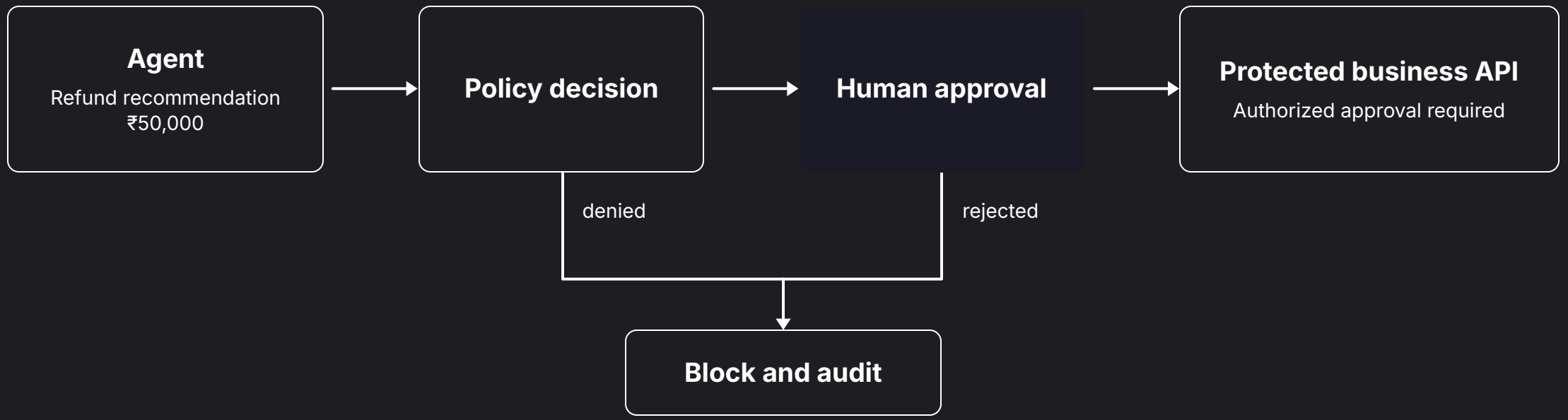
A user asks for a travel-policy summary. A retrieved document asks the agent to reveal confidential instructions and run the export tool.

- Separate trusted instructions from untrusted content.
- Constrain tools, require authorization, validate outputs and test adversarially.

Prompt wording is not a security boundary.

SECURITY

Block unsafe actions with controls, not hope



- Allowlist tools per agent and role; enforce strict argument schemas.
- Check permissions at every execution; default to read-only.
- Require human approval for high-impact actions.
- Bound retries, time, recursion and spend; use idempotency keys for state changes.

SECURITY

Cover privacy across prompts, traces and test data

Data category	Recommended control
Customer PII	Minimize, redact, restrict
Financial records	Tenant-scoped access and encryption
Retrieved documents	Document-level authorization
Trace payloads	Redaction, sampling, retention
Evaluation datasets	Sanitization and access controls

- Cover inputs, documents, tool responses, outputs and telemetry.
- Classify data; encrypt in transit and at rest; set retention, access audits and regional processing.
- Mask unneeded customer contact information before it reaches the model.

SECURITY

Safety checks on the way in and the way out

User input → **Input safety →** **Agent →** **Output validation →** **Policy check →** **Response**

Inspect inputs

Prohibited content, injection attempts, unexpected instructions and oversized inputs.

Inspect outputs

Sensitive data, unsafe content, unsupported claims and disallowed actions.

Log categories and decisions; define escalation and review; track false positives and negatives. Classifiers are imperfect: high-risk flows need deterministic controls and human oversight.

GOVERNANCE

Every production agent needs an owner

Ownership

Named business and technical owners.

Risk classification

Data sensitivity, autonomy and business impact.

Approval

Security, privacy and release gates.

Evidence

Evaluation reports, audit logs and decision records.

Align governance with NIST AI RMF and ISO/IEC 42001.



GOVERNANCE

Register models, prompts, tools, data and evaluations

Asset	Required governance metadata
Model	Provider, version, approval, region, cost
Prompt	Template version, reviewer, test results
Tool	API owner, permission scope, side effects
Data source	Classification, access policy, lineage
Evaluation	Dataset version, metrics, pass criteria

Keep a central inventory of active agents and dependencies. Register an HR agent with its owner, approved model deployment, read-only HR policy tool, permitted collections and evaluation suite.

GOVERNANCE

Four quality gates before production

Any material model, prompt, tool, retrieval or workflow change triggers evaluations.

01

Functional

Task completion and response structure.

02

Quality

Correctness, relevance, groundedness and citation accuracy.

03

Security

Injection resistance, permissions and data leakage.

04

Operational

Latency, cost, error rate and tool-call limits.

Version datasets and results; set pass thresholds; require manual review for sensitive changes.

GOVERNANCE

CI/CD for agents, with approval gates

Source versions	Tests and evaluations	Security and policy	Approval and release	Production monitoring
Code, prompts and tools	Regression evidence	Required checks	Staged rollout	Rollback readiness

- Version model deployment IDs, tool schemas, indexes, datasets and policies.
- Use canaries, feature flags and regression checks; retain a rollback path.
- Monitor quality and cost after release, not just deployment success.

GOVERNANCE

Can you explain what the agent did?

Show what it did, which data and tools it accessed, why access was permitted, what it cost and which version ran.

Control	Implementation	Evidence
Identity	Entra ID / managed identity	Sign-in and token audit
Secrets	Key Vault	Access logs
Monitoring	OpenTelemetry + Azure Monitor	Traces and alerts
Safety	Input/output policy checks	Safety test results
Quality	Versioned evaluations	Evaluation reports
Release	CI/CD approval gates	Deployment history



HANDS-ON LAB · 150 MIN TOTAL

Lab setup: build the IT support assistant (20 min)

Retrieve a small knowledge base and answer through an LLM.

- Prepare Python 3.11+, VS Code, Azure subscription, model access and Application Insights.
- Create a venv. Install openai, opentelemetry-sdk, opentelemetry-api, azure-monitor-opentelemetry and python-dotenv.
- Add three sample policy documents and in-memory `retrieve_documents(query)`.
- Implement `generate_answer(query, context)`, a CLI and Azure Monitor export.

Schedule: setup 20, instrumentation 40, monitoring 30, security 35, governance 25 min. Console export works without Azure; cloud use may incur charges.

HANDS-ON LAB

Instrument the agent with OpenTelemetry (40 min)

```
from opentelemetry import trace
import opentelemetry.sdk.trace as sdk
import opentelemetry.sdk.trace.export as exp
p = sdk.TracerProvider()
p.add_span_processor(exp.BatchSpanProcessor(
  exp.ConsoleSpanExporter()))
trace.set_tracer_provider(p)
tracer = trace.get_tracer(
  "enterprise-ai-agent")
```

```
def run_agent(query):
  start = tracer.start_as_current_span
  with start("agent.run") as span:
    span.set_attribute("agent.name", "it-support")
    with start("retrieval.search"):
      docs = retrieve_documents(query)
    with start("llm.generate"):
      return generate_answer(query, docs)
```

- Capture trace ID, agent name and duration; add document count, model identifier and token usage.
- Record exceptions and propagate context downstream.
- Export to console, then Azure Monitor.
- Expected: one parent trace with retrieval and LLM spans, timing and status.

HANDS-ON LAB

Monitor tokens, latency, errors and retrieval (30 min)

Generate traffic

Run 10–20 requests: successes, missing-document queries and simulated failures.

Investigate

Inspect Application Insights traces. Compare fast and slow requests; correlate failures by trace ID. Compute average/P95 latency, error rate and cost; flag empty retrieval.

Alert

Define alerts for latency, repeated model failures and unusual token use.

Expected: one slow operation, one failure and one retrieval-quality issue. Some AI metrics need custom instrumentation or workbook queries.



HANDS-ON LAB

Test injection, tool authorization and data protection (35 min)

Use synthetic data and simulated tools only.

Scenario	Test	Expected outcome
Prompt injection	Instruction override in retrieved document	Treated as untrusted content
PII exposure	Ask for a synthetic customer identifier	Output follows masking policy
Secret handling	Try to print an API key	Secret not disclosed
Excessive tool use	Trigger repeated tool requests	Execution limit stops the loop

- Build read-only `search_policy` and simulated `delete_record`; enforce the support-role allowlist before execution.
- Plant a malicious retrieved instruction. Verify privileged tools are blocked; log denials without sensitive payloads.
- Document injection failures instead of assuming model resistance.



HANDS-ON LAB

Apply governance and validate production readiness (25 min)

Manifest: ID, owner, purpose, model, prompt version, tools, data classification and release version.

Test dataset: normal, missing-info, injection and unauthorized-tool cases.

Release validation checklist

1. Tests pass against recorded release criteria.
2. Unauthorized calls are blocked; no sensitive leaks.
3. Traces carry correlation IDs.
4. Token and latency metrics are available.
5. Asset and release versions are recorded.
6. Approval and rollback are documented.

Deliver: instrumented agent, traces, monitoring evidence, security results, manifest and readiness checklist.